

Cloud, and why training a model is an HPC problem

The last unit, and the one where everything so far turns out to be the same thing.

Dr. Abhijith Anandakrishnan · Assistant Professor
23AID304 · Odd Semester 2026-27

Someone else owns the hardware, and you rent it by the minute through an API.

Everything else about cloud follows from that.

Five characteristics, or it is just hosting

- On-demand self-service
- Broad network access
- Resource pooling
- Rapid elasticity
- Measured service

A dedicated server rented for a year fails **elasticity** and **measured service**.

That is hosting, not cloud.

Where the responsibility line falls

	on-prem	IaaS	PaaS	SaaS
apps	you	you	you	them
data	you	you	you	them
runtime	you	you	them	them
OS	you	you	them	them
virt	you	them	them	them
svrs	you	them	them	them

Plus two that now matter more than classical PaaS:

CaaS you supply an image

FaaS you supply a function, billed per invocation

Cloud is not automatically cheaper

● Cloud wins

- Variable, bursty demand
- Short projects
- Hardware you cannot justify buying
- Try 8 H100s for four hours

● Owning wins

- Steady, high utilisation
- 4x A100 node $\approx$ \$2.90/hr owned
- Same instance $\approx$ \$12-15/hr rented
- Break-even near **25% utilisation**

Which is exactly why asaicompute exists.

And why a deadline burst should still be rented.

What catches people

EGRESS In is free. Out is \$0.08-0.12/GB. Ten TB of results costs ~\$1000 to retrieve.

That asymmetry **is** the lock-in mechanism.

SHARED RESPONSIBILITY Provider secures the cloud. **You** secure what you put in it.

Nearly every reported “cloud breach” is a customer misconfiguration.

Three kinds, one common mistake

TYPE	LATENCY	FOR
object	~100 ms	datasets, checkpoints
block	<1 ms	OS disks, databases
file	~1 ms	shared home dirs

Object storage is ****not**** a parallel filesystem. Every small read is an HTTP request. Stage down once, compute, write back.

What transfers, and what does not

- **Works well**

Embarrassingly parallel: parameter sweeps, Monte Carlo, hyperparameter search.

Barely communicates, so network quality hardly matters.

- **Needs care**

Tightly coupled MPI.

Virtual networking has higher and more **variable** latency than InfiniBand. Needs special HPC instances, at a price.

PART TWO

Training a large model is a distributed dense linear algebra problem with a collective in the inner loop.

Every performance idea in this course applies to it unchanged.

Where the time goes

1. Load a batch
2. **Forward** — matmuls
3. **Backward** — ~2x forward
4. **Gradient all-reduce**
5. Optimiser update

Steps 2 and 3 are cuBLAS and cuDNN. That is Unit 3.

Step 4 is `MPI_Allreduce` by another name. That is Unit 2.

Ring all-reduce

P GPUs in a ring. Split the gradient into P chunks.
P-1 reduce-scatter steps, then P-1 allgather steps.

Each GPU sends $2(P-1)/P \times N$ bytes $\rightarrow$ **2N, independent of P.**

That is why it scales: volume per GPU does not grow, only the step count.

UNIT 1'S RING TOPOLOGY

WILL THE NETWORK BE YOUR BOTTLENECK?

7B model, FP16 gradients = 14 GB

Ring all-reduce moves ~28 GB per GPU per step.

Over **NVLink** (600 GB/s): **47 ms**

Over **100 Gb IB** (12.5 GB/s): **2.2 s**

If a step's compute is ~200 ms:

within a node it overlaps and costs little.

across nodes it **dominates by 10x** and the GPUs idle.

That one calculation explains gradient compression, accumulation, overlap, and the entire interconnect market.

WHEN THE MODEL DOES NOT FIT

7B with Adam needs 112 GB

ITEM	BYTES/PARAM
FP16 weights	2
FP16 gradients	2
FP32 master	4
Adam m	4
Adam v	4
total	16

$7e9 \times 16 = \mathbf{112\ GB}$, before a single activation.

An 80 GB A100 cannot hold it.

THIS IS WHY ZERO AND FSDP EXIST

Three axes, usually combined

- **Tensor parallel**

Splits individual matrices.

All-reduce **inside every layer**. Very chatty → keep **inside a node**.

- **Pipeline parallel**

Splits layers into stages.

Activations between stages.
Introduces idle bubbles.

- **Sharded (ZeRO/FSDP)**

Splits optimiser state, gradients, parameters.

Gathers parameters just before use.

Training vs inference

- **Training**

Runs once, for weeks.

Metric: throughput. Large batches.

Bound by compute and interconnect.

- **Inference**

Runs constantly, for years.

Metric: latency and cost per request. Batch often 1.

Bound by **memory bandwidth**, almost always.

7B FP16, one token

Weights read: **14 GB** Arithmetic: **14 GFLOP**

$I = 14e9 / 14e9 = 1 \text{ FLOP/byte}$

A100 ridge ≈ 100 .

Two orders of magnitude to the left. The arithmetic units sit idle waiting for weights.

Quantisation, batching, KV caching, speculative decoding — every one of them is an attempt to **move right on the roofline**.

Unit 1's model, applied to the most commercially important computation of the decade.

And it gives the right answer.

Every layer is a unit of this course

```
PyTorch / JAX
DeepSpeed, FSDP, Megatron ← Unit 1
NCCL    cuDNN    cuBLAS    ← Units 2, 3
CUDA runtime and driver ← Unit 3
SLURM                                         ← Units 0, 2
GPUs, NVLink, InfiniBand ← Unit 1
```

That is why this unit is last.

Nothing here is new. It is the same handful of ideas at a larger scale.

What is the ceiling, and how close am I to it?

Almost every performance decision in your career will be a version of that question.